

Algorithms

Divide and Conquer

Dr. Mudassir Shabbir

LUMS University

March 3, 2026

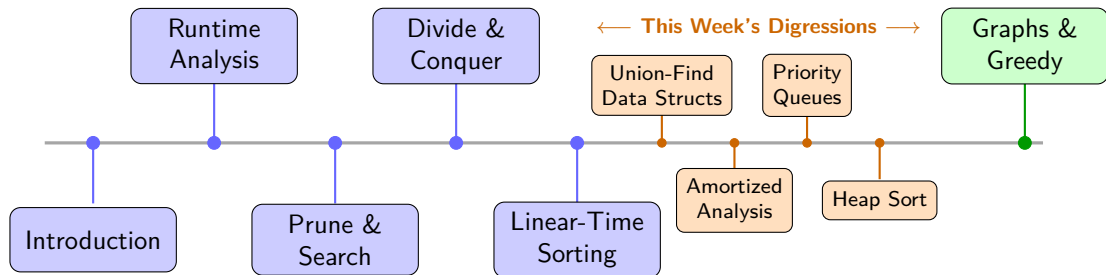


Announcements

- HW 3 is due on [Feb 09](#) at **11:59 AM**.
- HW 4 is out - no submission required.



Recap & What's Next



Data Structures

Union-Find & Priority Queues



Algorithms vs. Data Structures

Algorithm

Takes an **input**, performs a computation, returns an **output**.



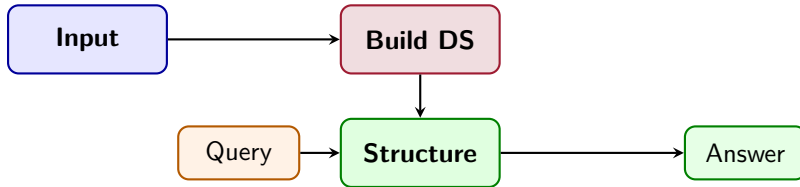
Example: MergeSort takes a list, returns a sorted list.



Algorithms vs. Data Structures

Data Structure

Takes an **input**, converts it into a **well-organised format** so that future **queries** are answered efficiently.



Algorithms vs. Data Structures

Algorithm

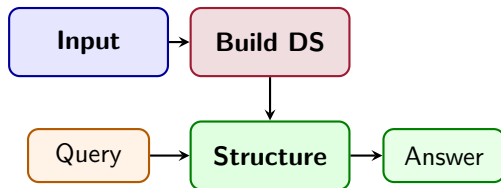
Takes an **input**, performs a computation, returns an **output**.



Example: MergeSort takes a list, returns a sorted list.

Data Structure

Takes an **input**, converts it into a **well-organised format** so that future **queries** are answered efficiently.



Key idea: Pay a one-time *preprocessing cost* so that each query is cheap.



Data Structures

Example: Phone Book

Raw data: 1 million (name, number) pairs.

Structure: Sort alphabetically.

Query: Find Alice's number.

→ Binary search: $O(\log n)$
instead of scanning: $O(n)$.

Sorted Phone Book

Alice	0300-111
Bob	0321-222
Carol	0331-333
Dave	0312-444
Eve	0345-555

↑ binary search



Union-Find

(Disjoint Set Union)



Are These Components Connected?

Scenario

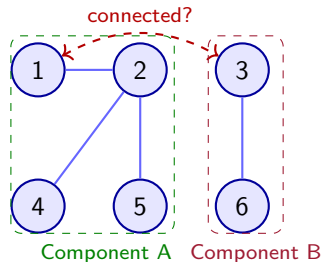
You manage a large computer network. Cables are added one by one. At any point, someone asks: “**Are computers u and v in the same connected component?**”

We need two operations:

- $\text{Union}(u, v)$: connect computers u and v
- $\text{Find}(u)$: which component is u in?

Same component? Check if $\text{Find}(u) = \text{Find}(v)$.

Application: This is exactly what Kruskal's MST algorithm needs: **detect if adding an edge would create a cycle!**



Union-Find

Start: 6 cities, each isolated. $\text{Find}(i) = i$ for all i .

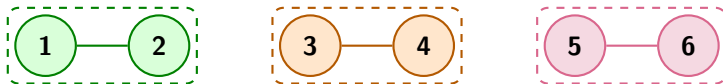


Union-Find

Start: 6 cities, each isolated. $\text{Find}(i) = i$ for all i .



After $\text{Union}(1, 2)$, $\text{Union}(3, 4)$, $\text{Union}(5, 6)$:

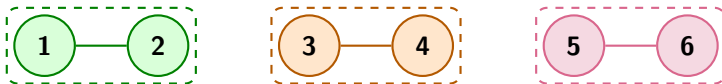


Union-Find

Start: 6 cities, each isolated. $\text{Find}(i) = i$ for all i .



After $\text{Union}(1, 2)$, $\text{Union}(3, 4)$, $\text{Union}(5, 6)$:



$\text{Find}(1) = \text{Find}(2)$ ✓ $\text{Find}(1) \neq \text{Find}(3)$ ✗ $\text{Find}(5) = \text{Find}(6)$ ✓



Brute Force Approach

Idea: Keep an array $\text{id}[i] = \text{component label of element } i$.

Find(x)

Just return $\text{id}[x]$.

Union(x, y)

Let $a = \text{id}[x]$, $b = \text{id}[y]$.

Scan entire array: for every i ,

if $\text{id}[i] = b$, set $\text{id}[i] \leftarrow a$.

Example: Union(3,5)

Before:

index	1	2	3	4	5	6
id	1	2	3	4	5	5

After (relabel all 3 $\rightarrow$ 5):

index	1	2	3	4	5	6
id	1	2	5	4	5	5



Brute Force Approach

Idea: Keep an array $\text{id}[i] = \text{component label of element } i$.

Find(x) $O(1)$

Just return $\text{id}[x]$.

Union(x, y) $O(n)$

Let $a = \text{id}[x]$, $b = \text{id}[y]$.

Scan entire array: for every i ,

if $\text{id}[i] = b$, set $\text{id}[i] \leftarrow a$.

Example: Union(3,5)

Before:

index	1	2	3	4	5	6
id	1	2	3	4	5	5

After (relabel all 3 $\rightarrow$ 5):

index	1	2	3	4	5	6
id	1	2	5	4	5	5



Brute Force Approach

Idea: Keep an array $\text{id}[i] = \text{component label of element } i$.

Find(x) $O(1)$

Just return $\text{id}[x]$.

Union(x, y) $O(n)$

Let $a = \text{id}[x]$, $b = \text{id}[y]$.

Scan entire array: for every i ,

if $\text{id}[i] = b$, set $\text{id}[i] \leftarrow a$.

Example: Union(3,5)

Before:

index	1	2	3	4	5	6
id	1	2	3	4	5	5

After (relabel all 3 $\rightarrow$ 5):

index	1	2	3	4	5	6
id	1	2	5	4	5	5

Problem

m union operations on n elements $\Rightarrow O(mn)$ total time. Far too slow for large graphs!

Better Idea: Forest of Trees

Represent each component as a rooted tree.

Each node stores a *parent pointer*. The root is the component **representative**.

Find(x)

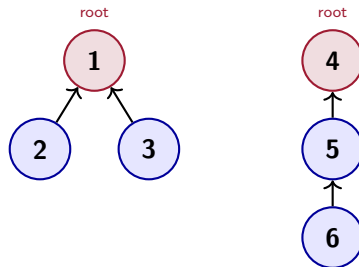
Follow parent pointers until you reach the root.
Return the root.

Union(x, y)

Find roots $r_x = \text{Find}(x)$, $r_y = \text{Find}(y)$.

If $r_x \neq r_y$: set $\text{parent}[r_x] \leftarrow r_y$.

Find(6): $6 \rightarrow 5 \rightarrow 4$. Returns **4**.



(depth = path length to root)



Better Idea: Forest of Trees

Represent each component as a rooted tree.

Each node stores a *parent pointer*. The root is the component **representative**.

Find(x)

Follow parent pointers until you reach the root.
Return the root.

Union(x, y)

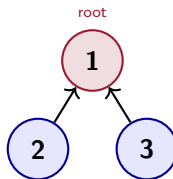
Find roots $r_x = \text{Find}(x)$, $r_y = \text{Find}(y)$.

If $r_x \neq r_y$: set $\text{parent}[r_x] \leftarrow r_y$.

Find(6): $6 \rightarrow 5 \rightarrow 4$. Returns **4**.

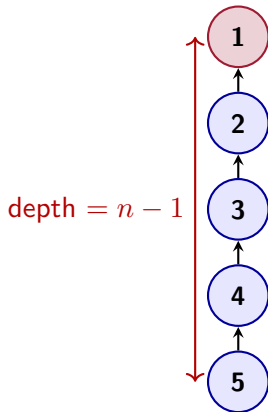
(depth = path length to root)

Problem: Chains can form! Find could degrade to $O(n)$.



The Problem: Trees Can Become Chains

Worst case for naive union: always attach bigger tree under smaller root.



Find(5) must traverse **4 edges**.

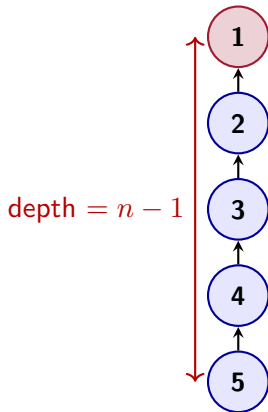
With n elements: $O(n)$ per Find!

m queries $\Rightarrow O(mn)$
just as bad as brute force.



The Problem: Trees Can Become Chains

Worst case for naive union: always attach bigger tree under smaller root.



Find(5) must traverse **4 edges**.

With n elements: $O(n)$ per Find!

m queries $\Rightarrow O(mn)$
just as bad as brute force.

Fix 1: Union by Rank

Fix 2: Path Compression

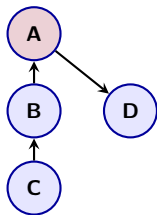
Together: nearly $O(1)$ per op!



Optimization 1: Union by Rank

Idea: Always attach the *shorter* tree under the root of the *taller* tree.
Track $\text{rank}[i]$ = upper bound on height of the subtree rooted at i .

Bad (naive): attach taller under shorter

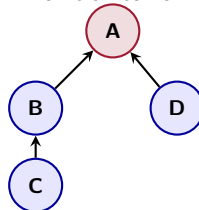


height = 3 (chain grows!)

Key Property

With union by rank, tree height is at most $\lfloor \log_2 n \rfloor$.
 $\Rightarrow$ Find costs $O(\log n)$ per operation.

Good (union by rank): attach shorter under taller

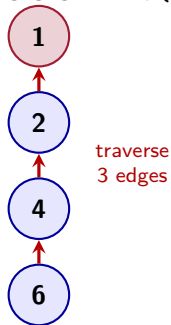


height = 2 (balanced)

Optimization 2: Path Compression

Idea: While executing $\text{Find}(x)$, make every node on the path point *directly* to the root.

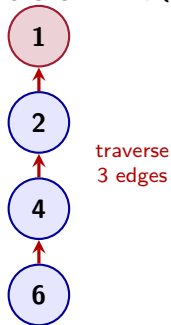
Before: Find(6)



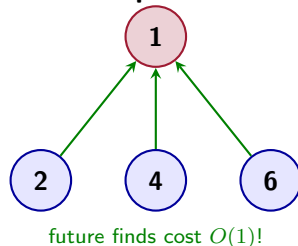
Optimization 2: Path Compression

Idea: While executing $\text{Find}(x)$, make every node on the path point *directly* to the root.

Before: Find(6)



After: all point to root



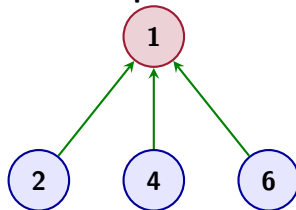
Optimization 2: Path Compression

Idea: While executing $\text{Find}(x)$, make every node on the path point *directly* to the root.

Implementation (recursive two-pass)

```
1: function Find( $x$ ):  
2: if parent[ $x$ ]  $\neq x$  then  
3:   parent[ $x$ ]  $\leftarrow$  Find(parent[ $x$ ])  
4: end if  
5: return parent[ $x$ ]
```

After: all point to root



future finds cost $O(1)$!

Union-Find: Full Algorithm

Initialize(n)

```
for  $i = 1$  to  $n$  do  
     $\text{parent}[i] \leftarrow i$   
     $\text{rank}[i] \leftarrow 0$   
end for
```

Find(x) [path compression]

```
if  $\text{parent}[x] \neq x$  then  
     $\text{parent}[x] \leftarrow \text{Find}(\text{parent}[x])$   
end if  
return  $\text{parent}[x]$ 
```

Union(x, y) [by rank]

```
 $rx \leftarrow \text{Find}(x); \quad ry \leftarrow \text{Find}(y)$   
if  $rx = ry$  then  
    return  
end if  
if  $\text{rank}[rx] < \text{rank}[ry]$  then  
    swap  $rx, ry$   
end if  
 $\text{parent}[ry] \leftarrow rx$   
if  $\text{rank}[rx] = \text{rank}[ry]$  then  
     $\text{rank}[rx] += 1$   
end if
```

Union-Find: What Do the Optimizations Buy Us?

Runtime with both optimizations

$O(\alpha(n))$ amortized per operation, where α is the **inverse Ackermann function**.

For all practical n : $\alpha(n) \leq 4$. **Essentially $O(1)$.**

The two optimizations contribute independently:

Optimization	Find	m ops on n elements
Neither	$O(n)$	$O(mn)$
Union by Rank only	$O(\log n)$	$O(m \log n)$
Path Compression only	$O(\log n)$ amortised	$O(m \log n)$
Both	$O(\alpha(n))$	$O(m \alpha(n))$

We now prove the $O(\log n)$ bound for **Union by Rank alone**.



Union by Rank: $O(\log n)$ Proof

Key Lemma

A tree whose root has rank k contains **at least** 2^k **nodes**.

Proof by induction on k :

Base ($k = 0$): Every node initialises with rank 0 and is a single-node tree. $1 \geq 2^0$. ✓

Step ($k \geq 1$): Rank increases by 1 in **Union** *only* when both roots have equal rank $k - 1$. By the induction hypothesis each subtree has $\geq 2^{k-1}$ nodes, so the merged tree has

$$\geq 2^{k-1} + 2^{k-1} = 2^k \text{ nodes. } \checkmark$$

Corollary: $\text{depth} \leq \log_2 n$

The entire forest has n nodes, so no root can have rank $> \log_2 n$. Since ranks are strictly increasing along every path to the root:

$$\text{height of any tree} \leq \log_2 n \Rightarrow \text{Find} \in O(\log n).$$

Union-Find: What Do the Optimizations Buy Us?

Runtime with both optimizations

$O(\alpha(n))$ **amortized per operation**, where α is the **inverse Ackermann function**.

For all practical n : $\alpha(n) \leq 4$. **Essentially $O(1)$.**

The two optimizations contribute independently:

Optimization	Find	m ops on n elements
Neither	$O(n)$	$O(mn)$
Union by Rank only	$O(\log n)$	$O(m \log n)$
Path Compression only	$O(\log n)$ amortised	$O(m \log n)$
Both	$O(\alpha(n))$	$O(m \alpha(n))$

We now prove the $O(\log n)$ bound for **Union by Rank alone**.



Amortized Analysis: Motivation

Problem: worst-case per-operation analysis can be overly pessimistic.

Naive worst-case reasoning

If one push to a dynamic array can cost $O(n)$ (when doubling), we might conclude n pushes cost $O(n^2)$.

This is **too pessimistic**: expensive operations are rare and get “paid for” by cheap ones.



Amortized Analysis: Motivation

Problem: worst-case per-operation analysis can be overly pessimistic.

Naive worst-case reasoning

If one push to a dynamic array can cost $O(n)$ (when doubling), we might conclude n pushes cost $O(n^2)$.

This is **too pessimistic**: expensive operations are rare and get “paid for” by cheap ones.

Amortized analysis

Bound the **total** cost of m operations, then divide by m .

$$\text{Amortized cost per op} = \frac{\text{total cost}}{m}$$

Worst-case sequence, not average-case - the bound holds always.



Amortized Analysis

Three standard methods

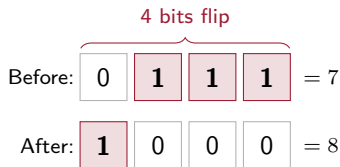
- 1 **Aggregate:** compute total cost $T(m)$ directly; amortized $= T(m)/m$.
- 2 **Accounting (Banker's):** assign credits to operations; overcharges pay for future costs.
- 3 **Potential:** define $\Phi(\text{state})$; amortized cost $= \text{actual} + \Delta\Phi$.



Binary Counter: Setup & Worst Case

Structure: a k -bit counter, initially $0 \cdots 0$. Increment adds 1 in binary with carry propagation. **Cost** = number of bits flipped.

Worst case - an all-ones counter:



In general, incrementing $\underbrace{011 \cdots 1}_k$ flips all k bits.

Naive worst-case bound

One Increment can flip k bits.

$\Rightarrow m$ increments cost $\leq mk$ flips.

Bound: $O(mk)$.

Is this tight?

Can every increment really cost k ?

No. To flip all k bits, the counter must first reach $\underbrace{1 \cdots 1}_k$, which requires $2^k - 1$

cheap increments first.

Binary Counter: Aggregate Analysis

Key observation: bit i (counting from 0) can flip at most once every 2^i increments, because it only toggles when all lower bits carry.

Flip frequency table

Bit i	After ops	Flips in m ops
0	every 1	m
1	every 2	$\lfloor m/2 \rfloor$
2	every 4	$\lfloor m/4 \rfloor$
3	every 8	$\lfloor m/8 \rfloor$
$\vdots$	$\vdots$	$\vdots$
i	every 2^i	$\lfloor m/2^i \rfloor$

Total flips summed over all bits:

$$T(m) = \sum_{i=0}^{k-1} \left\lfloor \frac{m}{2^i} \right\rfloor < m \sum_{i=0}^{\infty} \frac{1}{2^i} = 2m$$

The geometric series collapses to 2, regardless of k .

Result (Aggregate Method)

Total cost of m increments $< 2m = O(m)$.

Amortized cost per increment $= \frac{O(m)}{m} = O(1)$.

Binary Counter: The Charging Scheme

Goal: assign each Increment a fixed charge that covers *any* cascade of carry flips.

Credit rule: charge 2 per Increment



Binary Counter: The Charging Scheme

Goal: assign each Increment a fixed charge that covers *any* cascade of carry flips.

Credit rule: charge 2 per Increment

- 1 Pay 1 credit to flip the increment bit $0 \rightarrow 1$.



Binary Counter: The Charging Scheme

Goal: assign each Increment a fixed charge that covers *any* cascade of carry flips.

Credit rule: charge **2** per Increment

- 1 **Pay 1 credit** to flip the increment bit $0 \rightarrow 1$.
- 2 **Bank 1 credit** on that bit (label it “carry token”).



Binary Counter: The Charging Scheme

Goal: assign each Increment a fixed charge that covers *any* cascade of carry flips.

Credit rule: charge 2 per Increment

- 1 **Pay 1 credit** to flip the increment bit $0 \rightarrow 1$.
- 2 **Bank 1 credit** on that bit (label it “carry token”).
- 3 Each carry $1 \rightarrow 0$ is paid by the token *already stored on that bit* **free**.



Binary Counter: The Charging Scheme

Goal: assign each Increment a fixed charge that covers *any* cascade of carry flips.

Credit rule: charge 2 per Increment

- 1 **Pay 1 credit** to flip the increment bit $0 \rightarrow 1$.
- 2 **Bank 1 credit** on that bit (label it “carry token”).
- 3 Each carry $1 \rightarrow 0$ is paid by the token *already stored on that bit* **free**.

Invariant

At all times: **credits = number of 1-bits**.

Every 1-bit “owns” exactly one credit; we never carry without one.



Binary Counter: The Charging Scheme

Goal: assign each Increment a fixed charge that covers *any* cascade of carry flips.

Credit rule: charge 2 per Increment

- 1 Pay 1 credit to flip the increment bit $0 \rightarrow 1$.
- 2 Bank 1 credit on that bit (label it “carry token”).
- 3 Each carry $1 \rightarrow 0$ is paid by the token *already stored on that bit* free.

Invariant

At all times: **credits = number of 1-bits**.

Every 1-bit “owns” exactly one credit; we never carry without one.

Consequence

Credits ≥ 0 always, so the charge of 2 per increment is always sufficient.

Total charge = $2m \Rightarrow$ amortized cost = $O(1)$.

Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0

Counter at 0. No bits set, no credits held.



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1

000 → 001

Bit 0: 0 → 1

pay 1, bank 1 on bit 0.

Cost: 1 Credits: 1



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1

001 → 010

Bit 0: 1 → 0 use banked credit (*free!*)

Bit 1: 0 → 1 pay 1, bank 1 on bit 1.

Cost: 2 Credits: 1



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2

010 → 011

Bit 0: $0 \rightarrow 1$ pay 1, bank 1.

Cost: 1 Credits: 2



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2
4	100	3	1

011 → 100 (*carry cascade!*)

Bit 0: 1 → 0 use credit (*free!*)

Bit 1: 1 → 0 use credit (*free!*)

Bit 2: 0 → 1 pay 1, bank 1.

Cost: 3 Credits: 1



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2
4	100	3	1
5	101	1	2

100 $\rightarrow$ 101

Bit 0: 0 $\rightarrow$ 1 pay 1, bank 1.

Cost: 1 Credits: 2



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2
4	100	3	1
5	101	1	2
6	110	2	2

101 → 110

Bit 0: 1 → 0 use credit (*free!*)

Bit 1: 0 → 1 pay 1, bank 1.

Cost: 2 Credits: 2



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2
4	100	3	1
5	101	1	2
6	110	2	2
7	111	1	3

110 $\rightarrow$ 111

Bit 0: 0 $\rightarrow$ 1 pay 1, bank 1.

Cost: 1 Credits: 3



Binary Counter: Step-by-Step Trace

3-bit counter, charge 2 per increment.

Value	Binary	Cost	Balance
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2
4	100	3	1
5	101	1	2
6	110	2	2
7	111	1	3
Total (7 ops)		11	

All 7 increments done.

Total actual cost: 11

Total charged: $2 \times 7 = 14$

Credits never went negative. ✓

Amortized cost = $14/7 = 2 = O(1)$.



Binary Counter: Proving the Invariant

Invariant

After every Increment, $\text{Balance} = \text{number of 1-bits in the counter}$.

Base: 0 increments performed. Counter = $0 \cdots 0$, balance = 0, ones = 0. ✓

Step: Suppose the invariant holds (balance = ones). On the next Increment, let t bits flip $1 \rightarrow 0$ (carries), then exactly 1 bit flips $0 \rightarrow 1$.

	Balance	Ones
Before	c	c
t carries ($1 \rightarrow 0$)	$c - t$	$c - t$
1 flip ($0 \rightarrow 1$)	$c - t + 1$	$c - t + 1$
Net change	$1 - t$	$1 - t$

Both quantities change by $1 - t$, so balance = ones is preserved. ✓

Conclusion

Balance = ones ≥ 0 always, so the charge of 2 never over-drafts.

Total charge $\leq 2m$, amortized cost per op = $O(1)$.

Binary Counter: Accounting Method (Summary)

Idea: charge each Increment a fixed fee of **2 credits**. Use credits to pay for bit flips as they happen.

Credit rule

On each Increment:

- 1 **Pay 1 credit** to flip the target bit $0 \rightarrow 1$.
- 2 **Bank 1 credit** on that bit as a “future carry” token.
- 3 Every $1 \rightarrow 0$ carry-flip is paid by the credit already sitting on that bit.

Invariant: every bit currently set to 1 holds exactly 1 credit.

Value	Binary	Actual	Credits
0	000	—	0
1	001	1	1
2	010	2	1
3	011	1	2
4	100	3	1
5	101	1	2
6	110	2	2
7	111	1	3
Total (7 ops)		11	



Potential Method: The Framework

Idea: assign a *potential* $\Phi(D)$ to every state D of the data structure. Think of Φ as stored energy that cheap operations build up and expensive ones drain.

Amortized cost via potential

$$\hat{c}_i = c_i + \underbrace{\Phi(D_i) - \Phi(D_{i-1})}_{\Delta\Phi_i} \quad (\text{actual cost} + \text{change in potential})$$

Requirements on Φ

- 1 $\Phi(D_0) = 0$ (no stored energy at start)
- 2 $\Phi(D_i) \geq 0$ for all i (never go negative)

$$\sum_{i=1}^m c_i \leq \sum_{i=1}^m \hat{c}_i$$

Why this works (telescoping)

$$\sum_{i=1}^m c_i = \sum_{i=1}^m \hat{c}_i - \underbrace{(\Phi_m - \Phi_0)}_{\geq 0} \leq \sum_{i=1}^m \hat{c}_i$$

So bounding $\hat{c}_i$ per operation bounds the **total** actual cost.

Potential Method: Choosing Φ

Choose Φ = number of 1-bits

$$\Phi(\text{counter}) = \#\{\text{bits currently set to 1}\}$$

Check requirements:

- $\Phi_0 = 0$ (counter starts at $0 \cdots 0$). ✓
- $\Phi_i \geq 0$ always (it is a count). ✓

Analysing one Increment:

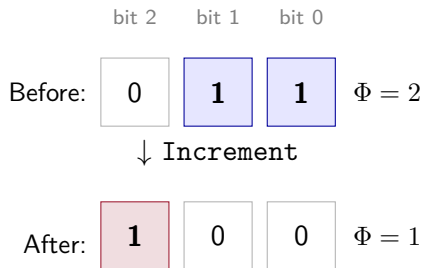
Let t = number of $1 \rightarrow 0$ carry-flips; then 1 bit flips $0 \rightarrow 1$.

Quantity	Expression	Intuition
Actual cost c_i	$t + 1$	t carries + 1 target flip
Change in potential $\Delta\Phi_i$	$1 - t$	gain 1 one-bit, lose t one-bits
Amortized $\hat{c}_i = c_i + \Delta\Phi_i$	$(t + 1) + (1 - t)$ $= 2$	t cancels out! constant, regardless of t



Potential Method: Concrete Example ($t = 2$)

Increment $011 \rightarrow 100$: all 3 bits flip, $t = 2$ carries.



Applying the formula

$$c_i = t + 1 = 2 + 1 = 3$$

$$\Delta\Phi_i = 1 - t = 1 - 2 = -1$$

$$\hat{c}_i = 3 + (-1) = \mathbf{2} \checkmark$$

3 bits flipped (expensive!), but Φ dropped by 1, releasing stored potential to absorb the extra work.

No matter how large t is, the drop in Φ covers the excess; amortized cost stays at 2.



Potential Method: Conclusion

Result for binary counter

Every Increment has $\hat{c}_i = 2$, so:

$$\sum_{i=1}^m c_i \leq \sum_{i=1}^m 2 - \underbrace{(\Phi_m - 0)}_{\geq 0} \leq 2m$$

Amortized cost per Increment = $O(1)$.

The potential method template

- 1 **Define** $\Phi(D)$ capturing “stored work” in state D .
- 2 **Verify** $\Phi_0 = 0$ and $\Phi \geq 0$.
- 3 **Compute** $\hat{c}_i = c_i + \Delta\Phi_i$ for a generic operation.
- 4 **Conclude**: total cost $\leq m \cdot \max_i \hat{c}_i$.

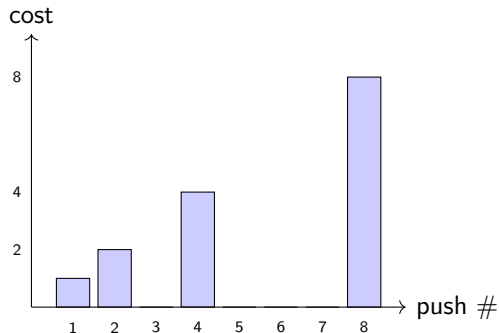
Where this appears again

The same technique proves the $O(\alpha(n))$ bound for Union-Find. The potential function there is far more complex, but the template is identical.

Amortized Analysis: Dynamic Array (See HW4.)

Operation: Push(x) to an array. When full, **double** the capacity and copy all elements.

Copying cost after n pushes (capacities 1, 2, 4, ...):



Aggregate: total work $\leq n + 2n = O(n)$.

Result

Total cost = $O(n)$.

Amortized cost per push = $O(1)$.

Copying events:

$$1 + 2 + 4 + \dots + \frac{n}{2} + n \leq 2n.$$



Find-Union Runtime Analysis

Approach	Find	Union	m ops on n elements
Flat array (brute force)	$O(1)$	$O(n)$	$O(mn)$
Naive trees	$O(n)$	$O(n)$	$O(mn)$
Union by rank only	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Path compression only	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Both (UF)	$O(\alpha(n))$	$O(\alpha(n))$	$O(m \alpha(n))$

Inverse Ackermann $\alpha(n)$

$\alpha(n) \leq 4$ for any n you'll ever see in practice ($n < 2^{65536}$).

Think of it as a constant for all engineering purposes.



Priority Queues

Always access the most important element first



Motivation: The Hospital Triage Problem

Scenario

Patients arrive at a hospital emergency room at unpredictable times. Each has a **severity score**. The doctor should always treat the **most critical patient** currently waiting; not the one who arrived first.

We need a structure supporting:

Priority Queue

Severity 9: Chest pain

← treat next!

Severity 6: Broken arm

Severity 4: Sprained ankle

Severity 2: Minor cut



Motivation: The Hospital Triage Problem

Scenario

Patients arrive at a hospital emergency room at unpredictable times. Each has a **severity score**. The doctor should always treat the **most critical patient** currently waiting; not the one who arrived first.

We need a structure supporting:

- $\text{Insert}(x, \text{priority})$: new patient arrives.

Priority Queue

Severity 9: Chest pain

← treat next!

Severity 6: Broken arm

Severity 4: Sprained ankle

Severity 2: Minor cut



Motivation: The Hospital Triage Problem

Scenario

Patients arrive at a hospital emergency room at unpredictable times. Each has a **severity score**. The doctor should always treat the **most critical patient** currently waiting; not the one who arrived first.

We need a structure supporting:

- $\text{Insert}(x, \text{priority})$: new patient arrives.
- $\text{ExtractMax}()$: treat the most critical patient.

Priority Queue

Severity 9: Chest pain

← treat next!

Severity 6: Broken arm

Severity 4: Sprained ankle

Severity 2: Minor cut



Motivation: The Hospital Triage Problem

Scenario

Patients arrive at a hospital emergency room at unpredictable times. Each has a **severity score**. The doctor should always treat the **most critical patient** currently waiting; not the one who arrived first.

We need a structure supporting:

- $\text{Insert}(x, \text{priority})$: new patient arrives.
- $\text{ExtractMax}()$: treat the most critical patient.
- $\text{PeekMax}()$: check who's next without removing.

Priority Queue

Severity 9: Chest pain

← treat next!

Severity 6: Broken arm

Severity 4: Sprained ankle

Severity 2: Minor cut



Motivation: The Hospital Triage Problem

Scenario

Patients arrive at a hospital emergency room at unpredictable times. Each has a **severity score**. The doctor should always treat the **most critical patient** currently waiting; not the one who arrived first.

We need a structure supporting:

- $\text{Insert}(x, \text{priority})$: new patient arrives.
- $\text{ExtractMax}()$: treat the most critical patient.
- $\text{PeekMax}()$: check who's next without removing.
- $\text{DecreaseKey}(x, \text{newPriority})$: patient's condition worsens.

Priority Queue

Severity 9: Chest pain

← treat next!

Severity 6: Broken arm

Severity 4: Sprained ankle

Severity 2: Minor cut



Brute Force Approaches



Brute Force Approaches

Unsorted Array

- Insert: append to end. $O(1)$.
- ExtractMax: scan all elements. $O(n)$.

Fast inserts, slow extraction.



Brute Force Approaches

Unsorted Array

- Insert: append to end. $O(1)$.
- ExtractMax: scan all elements. $O(n)$.

Fast inserts, slow extraction.

Sorted Array

- Insert: find position, shift. $O(n)$.
- ExtractMax: remove last element. $O(1)$.

Fast extraction, slow inserts.



Brute Force Approaches

Unsorted Array

- Insert: append to end. $O(1)$.
- ExtractMax: scan all elements. $O(n)$.

Fast inserts, slow extraction.

Sorted Array

- Insert: find position, shift. $O(n)$.
- ExtractMax: remove last element. $O(1)$.

Fast extraction, slow inserts.

Structure	Insert	ExtractMax
Unsorted array	$O(1)$	$O(n)$
Sorted array	$O(n)$	$O(1)$
Sorted linked list	$O(n)$	$O(1)$
Binary Heap	$O(\log n)$	$O(\log n)$



Brute Force Approaches

Unsorted Array

- Insert: append to end. $O(1)$.
- ExtractMax: scan all elements. $O(n)$.

Fast inserts, slow extraction.

Sorted Array

- Insert: find position, shift. $O(n)$.
- ExtractMax: remove last element. $O(1)$.

Fast extraction, slow inserts.

Structure	Insert	ExtractMax
Unsorted array	$O(1)$	$O(n)$
Sorted array	$O(n)$	$O(1)$
Sorted linked list	$O(n)$	$O(1)$
Binary Heap	$O(\log n)$	$O(\log n)$

Goal

Can we get $O(\log n)$ for **both**?

Yes! Use a **Binary Heap**.



Binary Heap: Structure



Binary Heap: Structure

A **binary heap** is a nearly-complete binary tree satisfying the **heap property**:

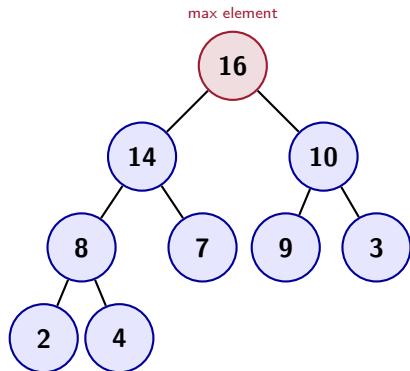
Every node's key is $\geq$ its children's keys. (max-heap)



Binary Heap: Structure

A **binary heap** is a nearly-complete binary tree satisfying the **heap property**:

Every node's key is $\geq$ its children's keys. (max-heap)



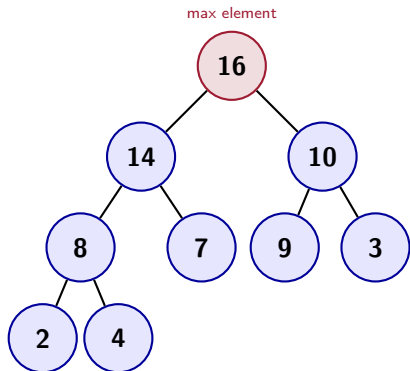
Binary Heap: Structure

A **binary heap** is a nearly-complete binary tree satisfying the **heap property**:

Every node's key is $\geq$ its children's keys. (max-heap)

Key properties:

- Height = $\lfloor \log_2 n \rfloor$
- Max is always at the **root**
- Stored in an array (no pointers needed!)



Binary Heap: Structure

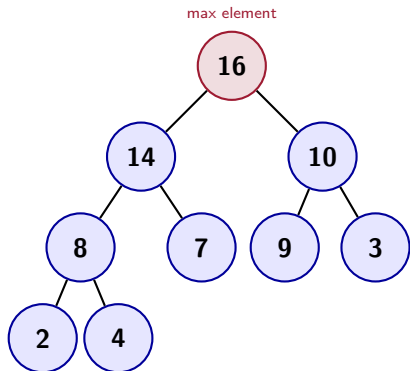
A **binary heap** is a nearly-complete binary tree satisfying the **heap property**:

Every node's key is $\geq$ its children's keys. (max-heap)

Key properties:

- Height = $\lfloor \log_2 n \rfloor$
- Max is always at the **root**
- Stored in an array (no pointers needed!)

Array representation:



i	1	2	3	4	5	6	7	8	9
val	16	14	10	8	7	9	3	2	4

Binary Heap: Structure

A **binary heap** is a nearly-complete binary tree satisfying the **heap property**:

Every node's key is $\geq$ its children's keys. (max-heap)

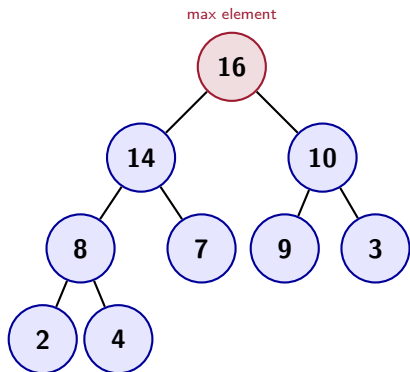
Key properties:

- Height = $\lfloor \log_2 n \rfloor$
- Max is always at the **root**
- Stored in an array (no pointers needed!)

Array representation:

i	1	2	3	4	5	6	7	8	9
val	16	14	10	8	7	9	3	2	4

left child = $2i$, right child = $2i + 1$, parent = $\lfloor i/2 \rfloor$.



Heap Operations: Insert



Heap Operations: Insert

Insert(x): add x at the end, then **sift up** (bubble up) to restore heap property.

Example: Insert 15 into the heap

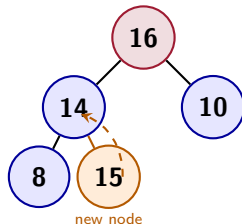


Heap Operations: Insert

Insert(x): add x at the end, then **sift up** (bubble up) to restore heap property.

Example: Insert 15 into the heap

Step 1: Append

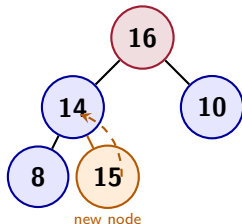


Heap Operations: Insert

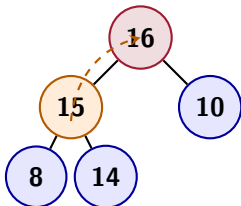
Insert(x): add x at the end, then **sift up** (bubble up) to restore heap property.

Example: Insert 15 into the heap

Step 1: Append



Step 2: Sift up ($15 > 14$, swap)

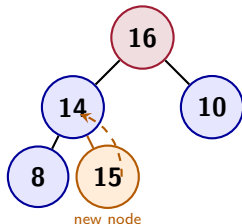


Heap Operations: Insert

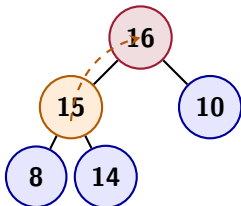
Insert(x): add x at the end, then **sift up** (bubble up) to restore heap property.

Example: Insert 15 into the heap

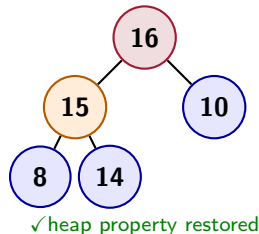
Step 1: Append



Step 2: Sift up ($15 > 14$, swap)



Step 3: $15 < 16$, stop

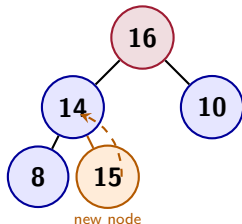


Heap Operations: Insert

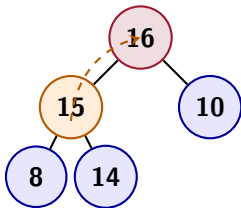
Insert(x): add x at the end, then **sift up** (bubble up) to restore heap property.

Example: Insert 15 into the heap

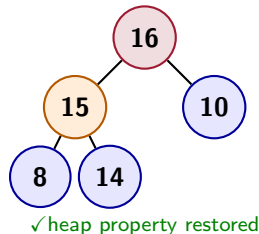
Step 1: Append



Step 2: Sift up ($15 > 14$, swap)



Step 3: $15 < 16$, stop



Cost: at most $\lfloor \log_2 n \rfloor$ swaps. $\Rightarrow O(\log n)$.



Heap Operations: ExtractMax



Heap Operations: ExtractMax

ExtractMax: remove root, put *last element* at root, then **sift down**.

Example: ExtractMax removes 16

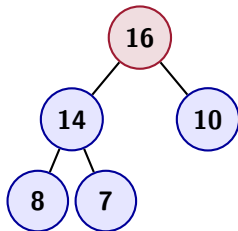


Heap Operations: ExtractMax

ExtractMax: remove root, put *last element* at root, then **sift down**.

Example: ExtractMax removes 16

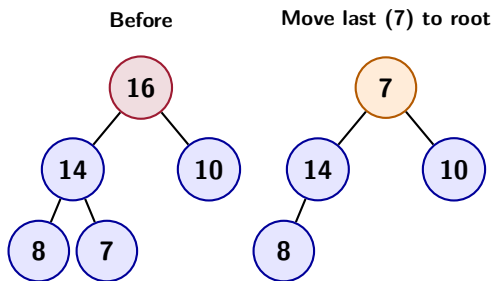
Before



Heap Operations: ExtractMax

ExtractMax: remove root, put *last element* at root, then **sift down**.

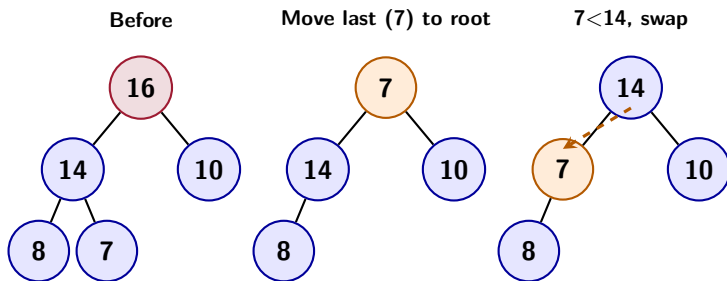
Example: ExtractMax removes 16



Heap Operations: ExtractMax

ExtractMax: remove root, put *last element* at root, then **sift down**.

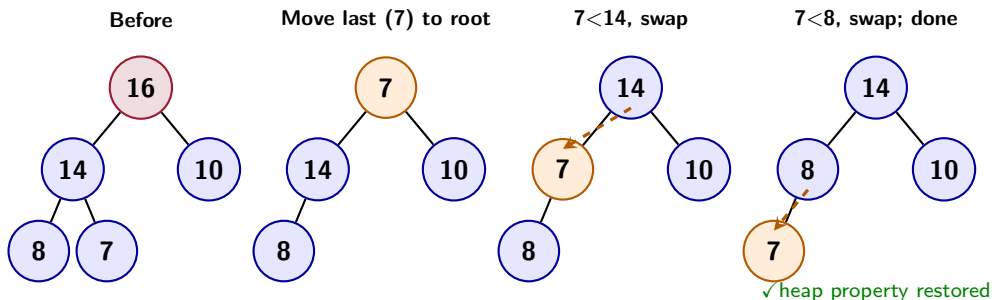
Example: ExtractMax removes 16



Heap Operations: ExtractMax

ExtractMax: remove root, put *last element* at root, then **sift down**.

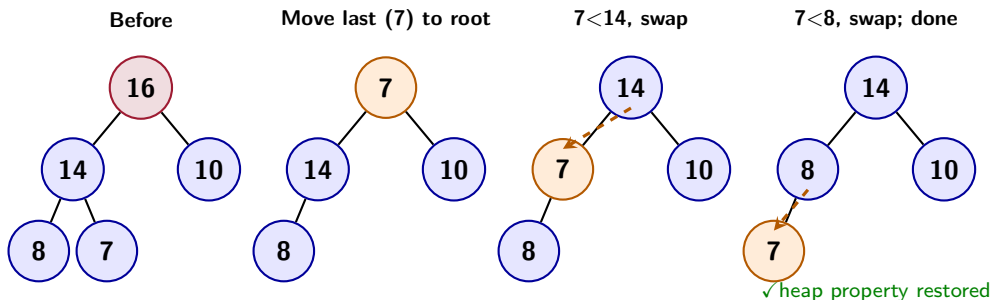
Example: ExtractMax removes 16



Heap Operations: ExtractMax

ExtractMax: remove root, put *last element* at root, then **sift down**.

Example: ExtractMax removes 16



Cost: at most $\lfloor \log_2 n \rfloor$ swaps. $\Rightarrow O(\log n)$.



Building a Heap: Heapify



Building a Heap: Heapify

Given an array of n elements, build a heap.

Naive approach: insert one-by-one. Cost: $O(n \log n)$.

Smarter: start from the last internal node ($\lfloor n/2 \rfloor$), sift down each node.



Building a Heap: Heapify

Given an array of n elements, build a heap.

Naive approach: insert one-by-one. Cost: $O(n \log n)$.

Smarter: start from the last internal node ($\lfloor n/2 \rfloor$), sift down each node.

BuildHeap

```
for  $i = \lfloor n/2 \rfloor$  downto 1 do  
    SiftDown( $i$ )  
end for
```



Building a Heap: Heapify

Given an array of n elements, build a heap.

Naive approach: insert one-by-one. Cost: $O(n \log n)$.

Smarter: start from the last internal node ($\lfloor n/2 \rfloor$), sift down each node.

BuildHeap

```
for  $i = \lfloor n/2 \rfloor$  downto 1 do  
    SiftDown( $i$ )  
end for
```

Novel Insight

Most nodes are near the bottom and sift down very little.

Maybe total work is $o(n \log n)$?



Building a Heap: Heapify

Given an array of n elements, build a heap.

Naive approach: insert one-by-one. Cost: $O(n \log n)$.

Smarter: start from the last internal node ($\lfloor n/2 \rfloor$), sift down each node.

BuildHeap

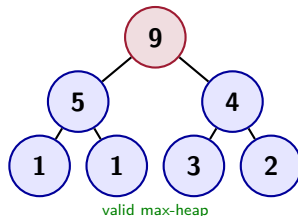
```
for  $i = \lfloor n/2 \rfloor$  downto 1 do  
    SiftDown( $i$ )  
end for
```

Novel Insight

Most nodes are near the bottom and sift down very little.

Maybe total work is $o(n \log n)$?

Example: Array $[3, 1, 4, 1, 5, 9, 2]$



BuildHeap: $O(n)$ Time Complexity — Proof



BuildHeap: $O(n)$ Time Complexity — Proof

Observe

- Nodes at height h : $\leq \lceil n/2^{h+1} \rceil$
- SiftDown on a height- h node: costs $O(h)$



BuildHeap: $O(n)$ Time Complexity — Proof

Observe

- Nodes at height h : $\leq \lceil n/2^{h+1} \rceil$
- SiftDown on a height- h node: costs $O(h)$

Height h	$\leq \# \text{nodes}$	Work
0 (leaves)	$n/2$	0
1	$n/4$	c
2	$n/8$	$2c$
3	$n/16$	$3c$
$\vdots$	$\vdots$	$\vdots$
$\lfloor \log n \rfloor$	1	$c \log n$

half the nodes are leaves (zero work), a **quarter** do 1 swap, etc. So on.

BuildHeap: $O(n)$ Time Complexity — Proof

Observe

- Nodes at height h : $\leq \lceil n/2^{h+1} \rceil$
- SiftDown on a height- h node: costs $O(h)$

Total work of BuildHeap:

$$T(n) \leq \sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot ch = \frac{cn}{2} \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \leq \frac{cn}{2} \underbrace{\sum_{h=0}^{\infty} \frac{h}{2^h}}_{=S}$$

Height h	$\leq \#nodes$	Work
0 (leaves)	$n/2$	0
1	$n/4$	c
2	$n/8$	$2c$
3	$n/16$	$3c$
$\vdots$	$\vdots$	$\vdots$
$\lfloor \log n \rfloor$	1	$c \log n$

half the nodes are leaves (zero work), a **quarter** do 1 swap, etc. So on.

BuildHeap: Evaluating $S = \sum_{h=0}^{\infty} h/2^h$

Recall: $T(n) \leq \frac{cn}{2} S$ where $S = \sum_{h=0}^{\infty} \frac{h}{2^h}$.



BuildHeap: Evaluating $S = \sum_{h=0}^{\infty} h/2^h$

Recall: $T(n) \leq \frac{cn}{2} S$ where $S = \sum_{h=0}^{\infty} \frac{h}{2^h}$.

Closed form via differentiation of the geometric series:

$$\sum_{h=0}^{\infty} x^h = \frac{1}{1-x} \xrightarrow{\frac{d}{dx}} \sum_{h=0}^{\infty} h x^{h-1} = \frac{1}{(1-x)^2} \xrightarrow{\times x} \sum_{h=0}^{\infty} h x^h = \frac{x}{(1-x)^2}.$$



BuildHeap: Evaluating $S = \sum_{h=0}^{\infty} h/2^h$

Recall: $T(n) \leq \frac{cn}{2} S$ where $S = \sum_{h=0}^{\infty} \frac{h}{2^h}$.

Closed form via differentiation of the geometric series:

$$\sum_{h=0}^{\infty} x^h = \frac{1}{1-x} \xrightarrow{\frac{d}{dx}} \sum_{h=0}^{\infty} h x^{h-1} = \frac{1}{(1-x)^2} \xrightarrow{\times x} \sum_{h=0}^{\infty} h x^h = \frac{x}{(1-x)^2}.$$

Evaluate at $x = \frac{1}{2}$:

$$S = \frac{1/2}{(1/2)^2} = 2.$$



BuildHeap: Evaluating $S = \sum_{h=0}^{\infty} h/2^h$

Recall: $T(n) \leq \frac{cn}{2} S$ where $S = \sum_{h=0}^{\infty} \frac{h}{2^h}$.

Closed form via differentiation of the geometric series:

$$\sum_{h=0}^{\infty} x^h = \frac{1}{1-x} \xrightarrow{\frac{d}{dx}} \sum_{h=0}^{\infty} h x^{h-1} = \frac{1}{(1-x)^2} \xrightarrow{\times x} \sum_{h=0}^{\infty} h x^h = \frac{x}{(1-x)^2}.$$

Evaluate at $x = \frac{1}{2}$:

$$S = \frac{1/2}{(1/2)^2} = 2.$$

Conclusion

$$T(n) \leq \frac{cn}{2} \cdot 2 = cn = O(n). \quad \blacksquare$$

Priority Queue: Complete Runtime Summary



Priority Queue: Complete Runtime Summary

Operation	Binary Heap	Description
BuildHeap	$O(n)$	Heapify an array
Insert	$O(\log n)$	Add element, sift up
ExtractMax	$O(\log n)$	Remove root, sift down
PeekMax	$O(1)$	Root of heap
DecreaseKey	$O(\log n)$	Update key, sift



Priority Queue: Complete Runtime Summary

Operation	Binary Heap	Description
BuildHeap	$O(n)$	Heapify an array
Insert	$O(\log n)$	Add element, sift up
ExtractMax	$O(\log n)$	Remove root, sift down
PeekMax	$O(1)$	Root of heap
DecreaseKey	$O(\log n)$	Update key, sift

For Prim's Algorithm

Prim's needs to repeatedly extract the minimum-weight edge crossing the current tree.

Using a binary heap (min-heap):

Total cost: $O((V + E) \log V)$.



Priority Queue: Complete Runtime Summary

Operation	Binary Heap	Description
BuildHeap	$O(n)$	Heapify an array
Insert	$O(\log n)$	Add element, sift up
ExtractMax	$O(\log n)$	Remove root, sift down
PeekMax	$O(1)$	Root of heap
DecreaseKey	$O(\log n)$	Update key, sift

For Prim's Algorithm

Prim's needs to repeatedly extract the minimum-weight edge crossing the current tree.

Using a binary heap (min-heap):

Total cost: $O((V + E) \log V)$.

For Dijkstra's Algorithm

Same structure: a min-priority queue of vertices ordered by tentative distance.

Total cost: $O((V + E) \log V)$.



Priority Queue in Action: Task Scheduler



Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.

Operation

Heap Contents (max at top)



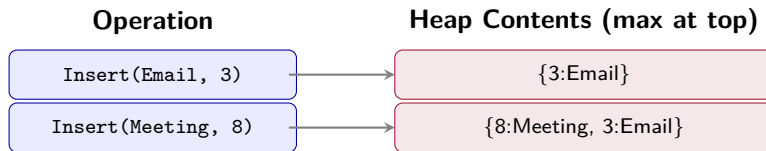
Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.



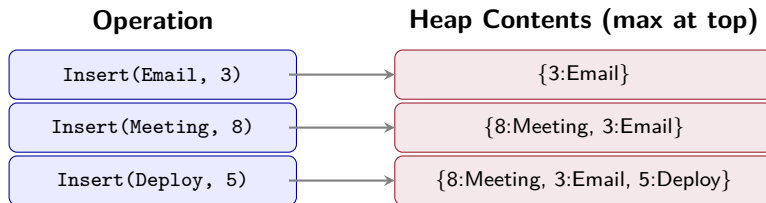
Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.



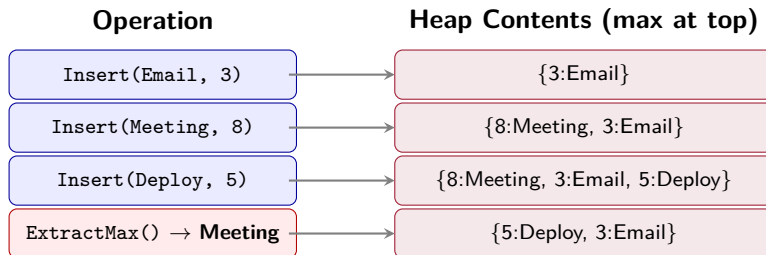
Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.



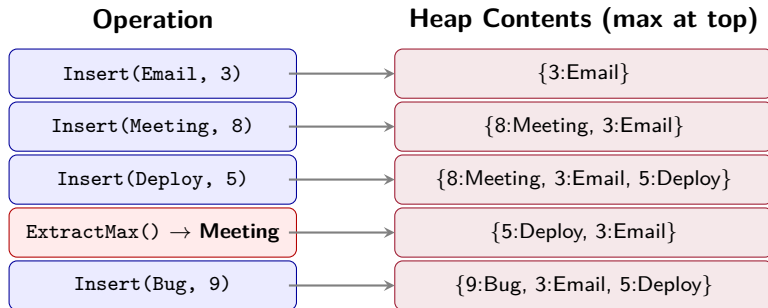
Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.



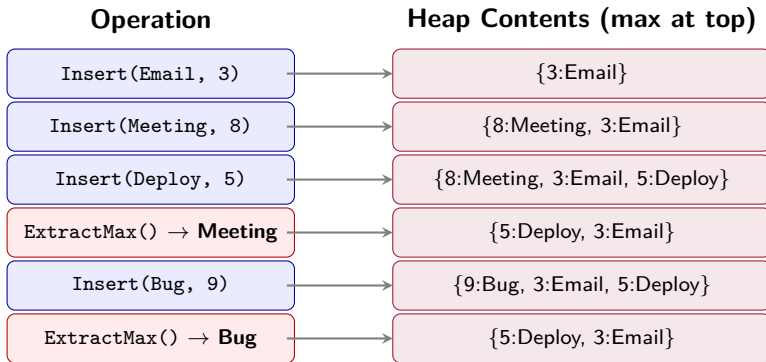
Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.



Priority Queue in Action: Task Scheduler

Tasks arrive with priorities. Always execute the highest priority task.



Application: Heap Sort



Application: Heap Sort

Heap Sort uses a priority queue to sort n elements in $O(n \log n)$ in-place.



Application: Heap Sort

Heap Sort uses a priority queue to sort n elements in $O(n \log n)$ in-place.

Algorithm

```
1: BuildHeap( $A$ )  $O(n)$ 
2: for  $i = n$  downto 2 do
3:   swap  $A[1] \leftrightarrow A[i]$ 
4:   Heap-size  $\leftarrow i - 1$ 
5:   SiftDown(1)  $O(\log n)$ 
6: end for
```



Application: Heap Sort

Heap Sort uses a priority queue to sort n elements in $O(n \log n)$ in-place.

Algorithm

```
1: BuildHeap( $A$ )  $O(n)$ 
2: for  $i = n$  downto 2 do
3:   swap  $A[1] \leftrightarrow A[i]$ 
4:   Heap-size  $\leftarrow i - 1$ 
5:   SiftDown(1)  $O(\log n)$ 
6: end for
```

Total:

$$O(n) + O(n \log n) = O(n \log n).$$

Space: $O(1)$ extra (in-place!).



Application: Heap Sort

Heap Sort uses a priority queue to sort n elements in $O(n \log n)$ in-place.

Algorithm

```
1: BuildHeap( $A$ )  $O(n)$ 
2: for  $i = n$  downto 2 do
3:   swap  $A[1] \leftrightarrow A[i]$ 
4:   Heap-size  $\leftarrow i - 1$ 
5:   SiftDown(1)  $O(\log n)$ 
6: end for
```

Trace on [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]:

After BuildHeap: **[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]**

Pass 1: swap $16 \leftrightarrow 1$, sift: **[14,...,16]**

Pass 2: swap $14 \leftrightarrow 1$, sift: **[10,...,14,16]**

$\vdots$

$\vdots$

Done: **[1,2,3,4,7,8,9,10,14,16]**

Total:

$O(n) + O(n \log n) = O(n \log n)$.

Space: $O(1)$ extra (in-place!).



Application: Heap Sort

Heap Sort uses a priority queue to sort n elements in $O(n \log n)$ in-place.

Algorithm

```
1: BuildHeap( $A$ )  $O(n)$ 
2: for  $i = n$  downto 2 do
3:   swap  $A[1] \leftrightarrow A[i]$ 
4:   Heap-size  $\leftarrow i - 1$ 
5:   SiftDown(1)  $O(\log n)$ 
6: end for
```

Total:

$O(n) + O(n \log n) = O(n \log n)$.

Space: $O(1)$ extra (in-place!).

Trace on [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]:

After BuildHeap: **[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]**

Pass 1: swap $16 \leftrightarrow 1$, sift: **[14,...,16]**

Pass 2: swap $14 \leftrightarrow 1$, sift: **[10,...,14,16]**

$\vdots$

$\vdots$

Done: **[1,2,3,4,7,8,9,10,14,16]**

Why Heap Sort matters

Like Merge Sort: $O(n \log n)$ worst-case.

Like Quick Sort: in-place.

Best of both worlds! (slower in practice.)

Summary: Data Structures for MST



Summary: Data Structures for MST

	Union-Find	Priority Queue
What it stores	Partition of elements into disjoint sets	Elements with priorities
Key ops	Find, Union	Insert, ExtractMin/Max
Implementation	Forest + path compression + union by rank	Binary heap
Cost per op	$O(\alpha(n)) \approx O(1)$	$O(\log n)$
Used by	Kruskal's MST	Prim's MST, Dijkstra



Summary: Data Structures for MST

	Union-Find	Priority Queue
What it stores	Partition of elements into disjoint sets	Elements with priorities
Key ops	Find, Union	Insert, ExtractMin/Max
Implementation	Forest + path compression + union by rank	Binary heap
Cost per op	$O(\alpha(n)) \approx O(1)$	$O(\log n)$
Used by	Kruskal's MST	Prim's MST, Dijkstra

Coming Up: Minimum Spanning Trees

With these two data structures in hand, we can implement Kruskal's and Prim's algorithm.

Minimum Spanning Tree



Why Minimum Spanning Trees? (Historical Motivation)

- During World War II, the Allies controlled railway infrastructure across Europe.
- Maintaining rail lines was expensive: fuel, guards, repairs, signaling.
- They needed a network that:
 - keeps all cities connected,
 - minimizes total maintenance cost.
- Question: **Which tracks should remain open?**



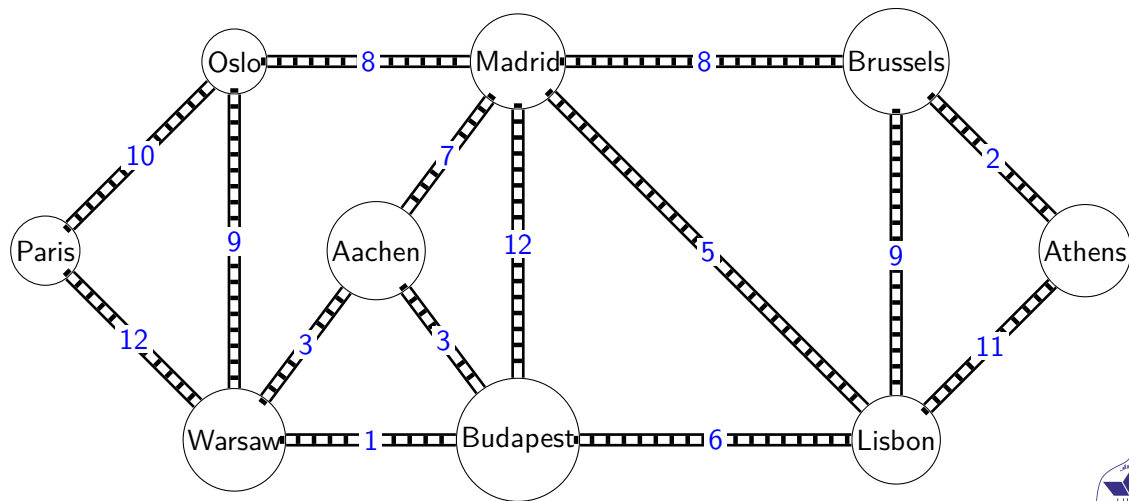
Why Minimum Spanning Trees? (Historical Motivation)

- During World War II, the Allies controlled railway infrastructure across Europe.
- Maintaining rail lines was expensive: fuel, guards, repairs, signaling.
- They needed a network that:
 - keeps all cities connected,
 - minimizes total maintenance cost.
- Question: **Which tracks should remain open?**

This is exactly the Minimum Spanning Tree problem.

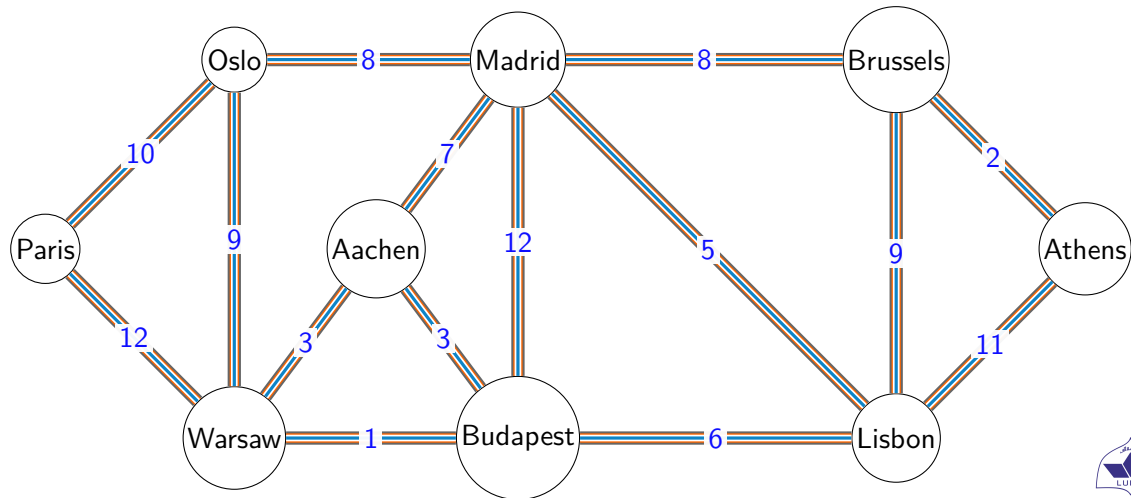


European rail network

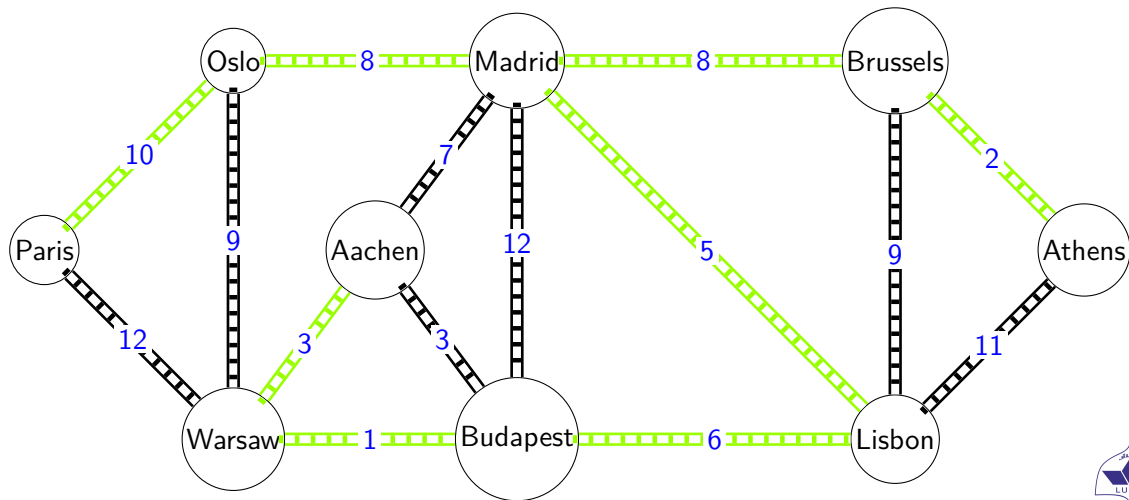


Goal: Keep cities connected while minimizing total cost.

A Modern Version: Fiber Network Design



Example Minimum Spanning Tree



Minimum Spanning Tree (MST) Problem

- Input: Weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}^+$.
- Output: Subset of edges $T \subseteq E$ such that:
 - T connects all vertices (spanning),
 - T contains no cycles (tree),
 - T has minimum total weight: $\sum_{e \in T} w(e)$.



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.

Question: How many subsets of edges are there?



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.

Question: How many subsets of edges are there?

$$2^{|E|}$$



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.

Question: How many subsets of edges are there?

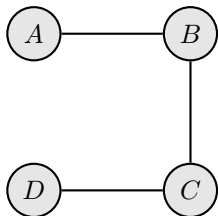
$$2^{|E|}$$

Conclusion: Brute force is exponential — infeasible.



Basic Fact 1: Adding an Edge to a Tree Creates a Cycle

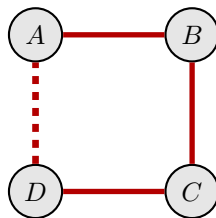
Tree (no cycle)



3 edges, 4 nodes. No cycle.

Unique path between every pair.

Add edge AD



Cycle $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$!

Now two paths exist between A and D .

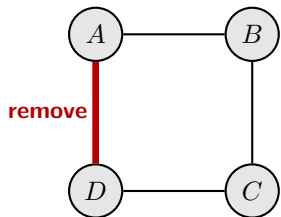
Fact. Adding *any* edge to a tree creates *exactly one* cycle.

The new edge provides a second path between its two endpoints, closing exactly one loop.



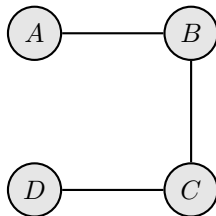
Basic Fact 2: Removing an Edge from a Cycle Keeps it Connected

Graph with cycle $A-B-C-D$



Remove edge DA .

Still connected!

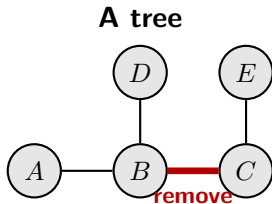


All nodes still reachable.
The result is a tree.

Fact. Removing any edge from a cycle leaves the graph *still connected*.
Every path that used that edge can be re-routed the other way around the cycle.

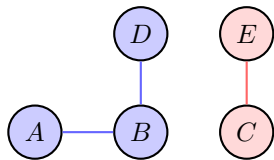


Basic Fact 3: Removing an Edge from a Tree Disconnects It



Remove edge BC .

Two components!



$\{A, B, D\}$ and $\{C, E\}$
are now disconnected.

Fact. Removing any edge from a tree creates *exactly two* connected components.

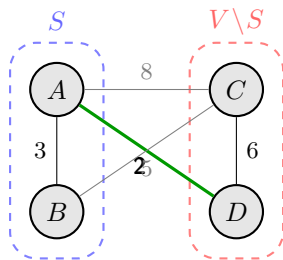
In a tree there is exactly one path between every pair of nodes. Remove the edge — all nodes on one side lose their only route to the other.

Key Insight: The Cut Property

Def. A **cut** $(S, V \setminus S)$ partitions vertices into two non-empty sets. An edge **crosses** the cut if its endpoints lie in different parts.

Cut Property. The unique min-weight edge crossing any cut belongs to every MST.

(Proof: same exchange argument — swap the crossing edge f in the MST for e^* .)



Crossing edges: AC (8), AD (2), BC (5).

Min crossing edge: AD (weight 2).

AD must be in every MST.

